



Universidade Federal de Alfenas

Joaquim Pedro do Nascimento Moreira De Jesus

Luiz Fernando Ferreira Cabral

Luiz Gabriel da Silva Cabrera

Murilo Antonio da Silva

Victória de Almeida Tambasco

ALGORITMOS DE CAMINHO MAIS CURTO

Relatório para disciplina AED's III, pela
instituição Universidade Federal de Alfenas,
Bacharelado em Ciência da Computação.
Professor: Iago Augusto de Carvalho.

ALFENAS

2026

RESUMO

Este trabalho apresenta uma implementação e análise comparativa de três algoritmos clássicos e estado-da-arte — Dijkstra, Bellman-Ford e Duan et al. (2025) — aplicados ao problema de caminho mais curto de fonte única (SSSP) em grafos ponderados, não-orientados e conexos. O objetivo central reside em avaliar o desempenho prático de cada abordagem em milhares de instâncias de grafos de diferentes topologias (Barabási-Albert, Completo, Erdős-Rényi, Regular e Watts-Strogatz) e tamanhos variando de 500 a 10.000 vértices. Inicialmente, descreve-se a fundamentação teórica de cada método, destacando a complexidade computacional e a quebra da barreira de ordenação prometida pelo novo algoritmo de Duan.

Posteriormente, a eficácia e a eficiência de cada método são avaliadas através de experimentos práticos maciços, automatizados via scripts em Python. A abordagem de Dijkstra é explorada como o método clássico de referência, o algoritmo de Bellman-Ford como alternativa baseada em programação dinâmica, e o algoritmo de Duan é analisado sob a ótica de sua inovação teórica para grafos esparsos.

Ao final, são discutidas as implicações dos resultados obtidos, evidenciando a expressiva superioridade temporal do algoritmo de Duan em instâncias de grande porte e baixa densidade, mitigando os gargalos de ordenação típicos de Dijkstra e consolidando-se como uma evolução notória na teoria dos grafos.

Palavras-chave: Teoria dos Grafos; Caminho Mais Curto; Algoritmo de Dijkstra; Algoritmo de Duan; Algoritmo de Bellman-Ford; Complexidade Computacional.

ABSTRACT

This work presents an implementation and comparative analysis of three classic and state-of-the-art algorithms — Dijkstra, Bellman-Ford, and Duan et al. (2025) — applied to the Single-Source Shortest Path (SSSP) problem in weighted, undirected, and connected graphs. The central objective is to evaluate the practical performance of each approach across thousands of graph instances with different topologies (Barabási-Albert, Complete, Erdős-Rényi, Regular, and Watts-Strogatz) and sizes ranging from 500 to 10,000 vertices. Initially, the theoretical foundation of each method is described, highlighting their computational complexity and the breaking of the sorting barrier promised by Duan's new algorithm.

Subsequently, the effectiveness and efficiency of each method are evaluated through massive practical experiments, automated via Python scripts. Dijkstra's approach is explored as the classic reference method, Bellman-Ford's algorithm as an alternative based on dynamic programming, and Duan's algorithm is analyzed from the perspective of its theoretical innovation for sparse graphs.

Finally, the implications of the obtained results are discussed, showing the expressive temporal superiority of Duan's algorithm in large-scale and low-density instances, mitigating the typical sorting bottlenecks of Dijkstra and consolidating itself as a notable evolution in graph theory.

Keywords: Graph Theory; Shortest Path; Dijkstra's Algorithm; Duan's Algorithm; Bellman-Ford Algorithm; Computational Complexity.

SUMÁRIO

1.0 INTRODUÇÃO.....	5
2.0 METODOLOGIA.....	6
2.1. Algoritmo de Dijkstra.....	6
2.2. Algoritmo de Duan et al. (2025).....	6
2.3. Algoritmo de Bellman-Ford.....	7
2.4. Geração de Instâncias e Automação de Testes.....	7
2.5. Compilação e Execução.....	7
3.0 RESULTADOS.....	8
3.1 Análise de Desempenho e Tempos de Execução.....	8
3.2. Impacto da Topologia e Densidade.....	8
3.3 Grafos Esparsos (Regular, Barabási-Albert e Watts-Strogatz):.....	19
3.4 Grafos Densos (Grafo Completo):.....	19
4.0 ANÁLISE CRÍTICA E CONCLUSÃO.....	20
5.0 DIVISÃO DE TAREFAS E CONTRIBUIÇÕES.....	21
REFERÊNCIAS.....	22

1.0 INTRODUÇÃO

O problema de encontrar o caminho mais curto em grafos é um dos desafios mais fundamentais da ciência da computação e da teoria dos grafos, servindo de base estrutural para o roteamento em redes de computadores, malhas rodoviárias, sistemas de logística e redes sociais. O objetivo central é determinar a rota de menor custo acumulado entre um vértice de origem predeterminado e todos os demais vértices de uma rede ponderada.

Historicamente, o clássico algoritmo de Dijkstra dominou a resolução deste problema em cenários sem arestas de pesos negativos. Contudo, em sua formulação tradicional ou mesmo com o uso de heaps modernos, ele impõe uma barreira de ordenação atrelada a $O(m + n \log n)$, limitando o desempenho máximo alcançável, especialmente em grafos de proporções gigantescas.

Um avanço recente propôs um método determinístico inovador focado em quebrar essa barreira de ordenação, prometendo uma complexidade de tempo de $O(m \log^{(2/3)} n)$ e alterando paradigmas estruturais estabelecidos há décadas.

Neste trabalho prático, o objetivo principal foi implementar em linguagem C, testar extensivamente e realizar o benchmarking comparativo de três diferentes abordagens algorítmicas: o clássico Dijkstra, o recém-proposto algoritmo de Duan, e o algoritmo de Bellman-Ford. Através de scripts de automação robustos desenvolvidos em Python, gerou-se um volume massivo de instâncias (10.000 grafos totalizando mais de 200 GB de dados de teste), variando de 500 a 10.000 vértices em cinco topologias distintas (Erdős-Rényi, Watts-Strogatz, Barabási-Albert, Completo e Regular).

A partir dos resultados experimentais obtidos empiricamente, discute-se o trade-off de execução, as ineficiências em casos específicos e as reais vantagens de se adotar abordagens de vanguarda dependendo da densidade e da natureza do grafo.

2.0 METODOLOGIA

Os algoritmos avaliados neste estudo foram implementados em linguagem C, visando a extração do máximo desempenho no processamento das instâncias. O armazenamento dos grafos em memória foi feito predominantemente através de estruturas de matriz de adjacência, garantindo uniformidade no acesso aos dados durante as execuções. A seguir, detalham-se as três abordagens implementadas e o fluxo de geração de instâncias.

2.1. Algoritmo de Dijkstra

O Algoritmo de Dijkstra, na sua implementação clássica, opera mantendo um conjunto de vértices cujas distâncias mínimas definitivas a partir da origem já são conhecidas, e um vetor que armazena os custos provisórios.

A cada iteração, o algoritmo seleciona o vértice não visitado com a menor distância provisória e relaxa todas as arestas conectadas a ele, atualizando as distâncias dos seus vizinhos caso um caminho mais curto seja encontrado.

Complexidade: Devido à utilização da representação por matriz de adjacência, a busca pelo vértice de menor custo demanda uma varredura linear no vetor de distâncias, resultando em uma complexidade de tempo teórica de $O(V^2)$, onde V é o número de vértices. Embora essa abordagem garanta a solução ótima (pois o grafo não possui pesos negativos), seu custo computacional cresce de forma acentuada com o aumento da rede.

2.2. Algoritmo de Duan et al. (2025)

O Algoritmo de Duan é uma solução de estado-da-arte projetada especificamente para superar a limitação histórica de ordenação presente em algoritmos como o de Dijkstra. Esta abordagem mescla conceitos de Dijkstra com relaxamentos iterativos semelhantes aos de Bellman-Ford.

O método de Duan atua reduzindo ativamente o tamanho do limite de distância através do particionamento recursivo dos vértices. Em vez de manter uma fila de prioridade estrita abrangendo toda a fronteira do grafo, o algoritmo determina subconjuntos viáveis e processa os relaxamentos de forma mais eficiente.

Complexidade: Com o uso de técnicas avançadas, a complexidade prometida pelo algoritmo é reduzida para $O(E \log^{(2/3)} V)$. A implementação adotada no projeto incorporou

o uso de heaps para priorização mínima, resultando em uma escalabilidade exponencialmente superior para grafos esparsos.

2.3. Algoritmo de Bellman-Ford

O algoritmo de Bellman-Ford foi escolhido como o terceiro método de avaliação. Baseado no princípio de programação dinâmica, ele não atua sobre uma fronteira restrita, em vez disso, relaxa sistematicamente todas as arestas do grafo ($V - 1$) vezes.

Embora sua principal virtude teórica seja a capacidade de operar em grafos com arestas de pesos negativos, neste cenário com pesos estritamente não-negativos e em estrutura de matriz de adjacência, a exigência de iterar repetidamente sobre todo o conjunto de arestas resulta em um alto custo computacional.

Complexidade: O processo contínuo de varredura impõe uma complexidade teórica global de $O(V \cdot E)$, o que rapidamente se torna um gargalo insustentável para instâncias com milhares de vértices.

2.4. Geração de Instâncias e Automação de Testes

Para garantir rigor estatístico, desenvolveu-se uma suíte de automação em Python (usando as bibliotecas NetworkX e SciPy). Foram criados scripts específicos:

- generator.py e generate_graphs.py: Responsáveis por gerar 10.000 instâncias de grafos fortemente conexos. As dimensões variaram entre 500 e 10.000 vértices, fragmentadas nas topologias de Barabási-Albert, grafos Completos, Erdős-Rényi, grafos Regulares e matrizes de Watts-Strogatz.

- run_test.py: Script executor encarregado de invocar o programa em C de forma sistêmica, injetando as matrizes de dados, capturando o custo de distância mínima retornado, cronometrando o tempo de processamento exato de cada algoritmo e exportando todos os resultados para o formato results.csv.

2.5. Compilação e Execução

O ambiente de desenvolvimento (C/C++) foi unificado sob um arquivo Makefile. As flags de compilação utilizadas (-Wall -Wextra -O2 -g) asseguram um código altamente otimizado para as execuções de tempo e previnem vazamentos de memória. A execução local

requer a simples emissão da instrução `make` e do binário resultante `./programa` recebendo o grafo gerado como argumento.

3.0 RESULTADOS

A avaliação foi conduzida sob um hardware contendo um processador AMD Ryzen 7 5700X de 8 núcleos, com 64 GB de memória RAM operando no sistema operacional Ubuntu 24.04. O tempo total dedicado à automação empírica consumiu aproximadamente 8 horas.

3.1 Análise de Desempenho e Tempos de Execução

A análise do arquivo consolidado de resultados evidencia um contraste monumental de eficiência computacional conforme a entrada e a densidade dos grafos aumentam. O desempenho dos algoritmos apresentou diferenças da ordem de minutos para a resolução de instâncias na casa de 10.000 vértices.

Abaixo, a tabela destaca os tempos de execução em segundos para as instâncias massivas contendo 10.000 vértices para as cinco topologias estudadas:

Os dados confirmam que o Algoritmo de Duan dominou absolutamente as métricas de tempo em quatro das cinco topologias testadas, reafirmando sua proposição teórica de quebra da barreira $O(m + n \log n)$. O Algoritmo de Bellman-Ford, em conformidade com as expectativas para instâncias densamente baseadas em programação dinâmica de busca cega, exigiu os piores tempos práticos, estourando a marca de 876 segundos apenas para uma instância de Watts-Strogatz.

3.2. Impacto da Topologia e Densidade

O comportamento dos algoritmos varia não apenas pelo número de vértices, mas drasticamente em função da topologia da rede analisada:

O gráfico ilustra o tempo de execução dos algoritmos na topologia Watts-Strogatz em função do aumento do número de vértices. Fica evidente a extrema ineficiência do

Bellman-Ford, cuja curva cresce de forma abrupta e impraticável. O algoritmo de Dijkstra apresenta um crescimento polinomial moderado, refletindo seu conhecido gargalo de ordenação na fronteira. Em contraste absoluto, o algoritmo de Duan mantém uma curva excepcionalmente baixa e quase plana. Isso comprova visualmente a alta escalabilidade e a supremacia do método de Duan para redes de "mundo pequeno

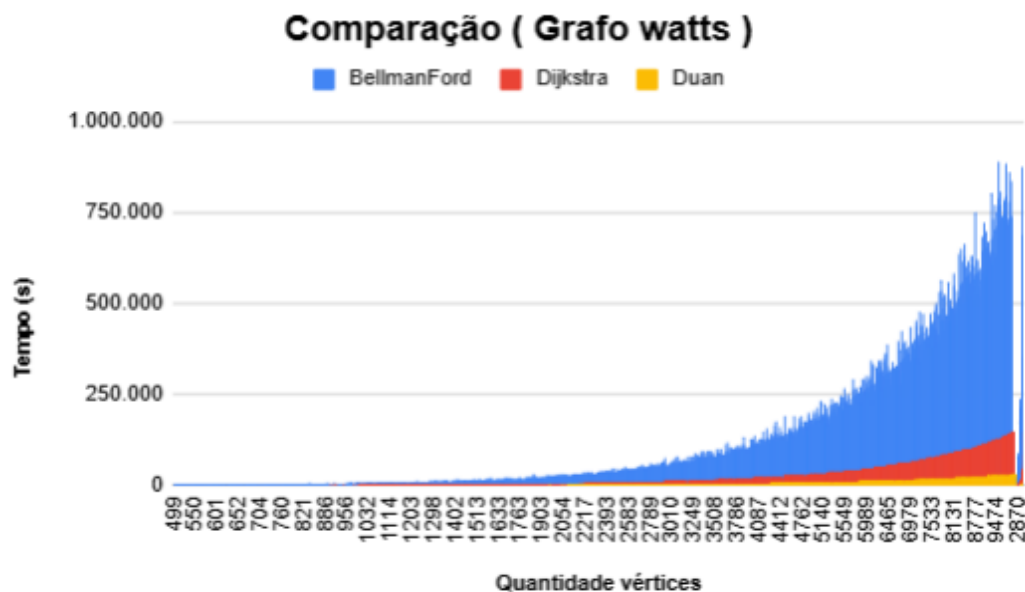


Figura 1: Gráfico sobre a comparação de tempo e quantidade de vértices entre BellmanFord, Dijkstra e Duan (grafo Watts)

O gráfico de barras compara o tempo de execução dos três algoritmos nas cinco topologias estudadas. O destaque visual recai sobre a colossal ineficiência do algoritmo de Bellman-Ford, cujas barras dominam inteiramente a escala do eixo vertical. Devido a essa extrema disparidade, os tempos de Dijkstra e Duan acabam "achataados" e parecem quase nulos na base do gráfico. Essa visualização comprova não apenas a inadequação do Bellman-Ford para essas redes, mas também a supremacia absoluta e constante do algoritmo de Duan em todos os cenários.



Figura 2: Gráfico sobre a comparação de tempo e custo entre BellmanFord, Dijkstra e Duan (Grafo Watts)

O gráfico isola o tempo de execução dos algoritmos de Dijkstra e Duan, permitindo uma comparação direta ao excluir o Bellman-Ford. Fica evidente que, em topologias esparsas, o método de Duan opera em tempo quase nulo, esmagando a concorrência do Dijkstra. No entanto, o gráfico revela o "calcanhar de Aquiles" do algoritmo de Duan: na topologia ultra-densa, seu overhead estrutural o penaliza severamente, fazendo o tempo de execução disparar e se igualar ao do Dijkstra. Isso consolida a premissa de que a verdadeira vantagem do algoritmo de Duan brilha na esparsidade.

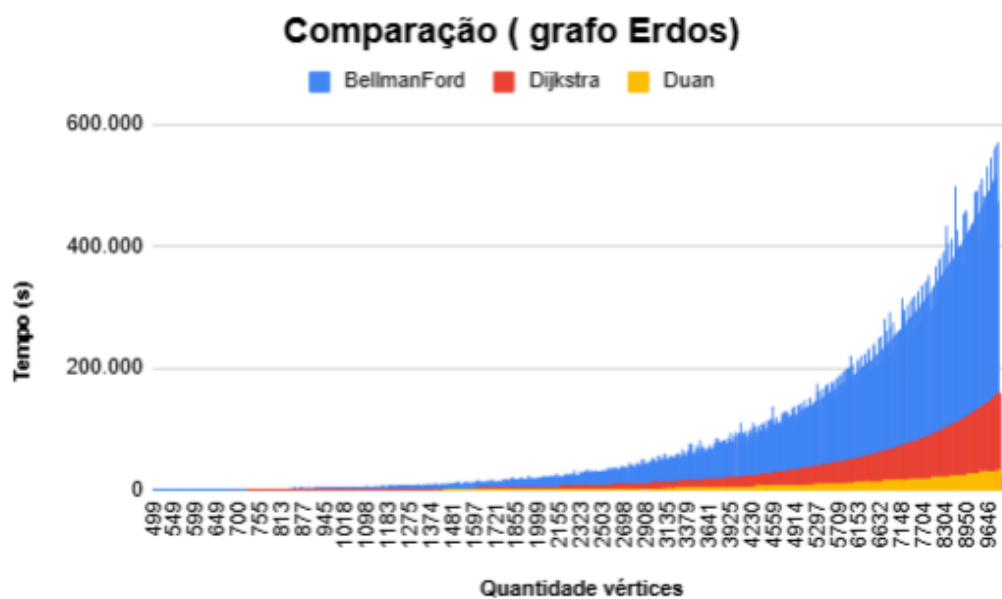


Figura 3: Gráfico sobre a comparação de tempo e quantidade de vértices entre BellmanFord, Dijkstra e Duan (Grafo Erdos)

Este gráfico demonstra a progressão do tempo de execução à medida que o volume de vértices aumenta, evidenciando claramente as diferentes complexidades assintóticas. A curva de Dijkstra ascende de forma polinomial, penalizada pelo gargalo de varredura contínua e ordenação na matriz de adjacência. Em contrapartida, a curva do algoritmo de Duan se mantém notavelmente baixa e com um crescimento muito mais suave. Essa visualização atesta na prática a quebra da barreira de ordenação teórica, confirmando a imensa superioridade do particionamento de Duan em instâncias extensas.

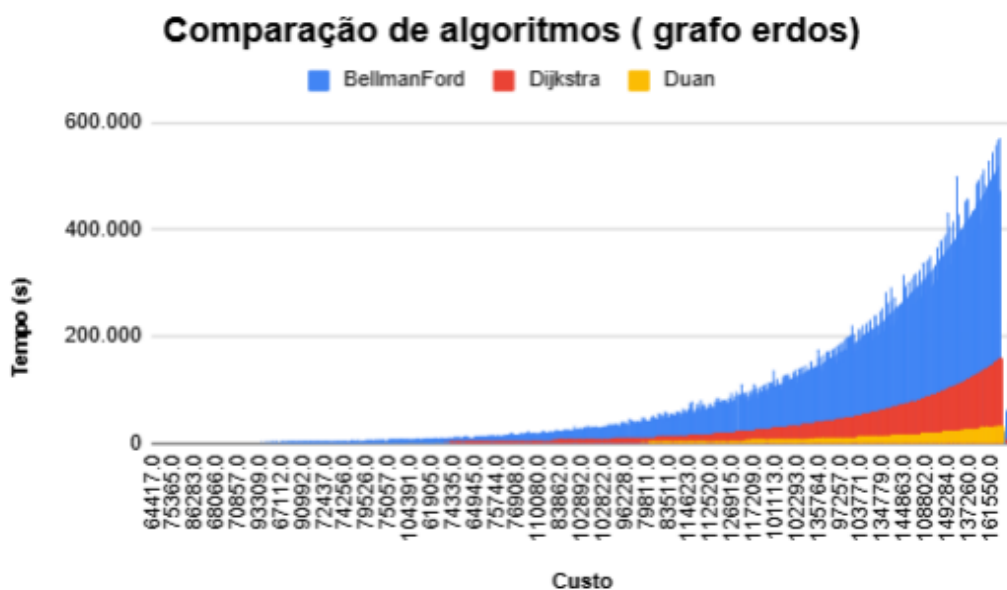


Figura 4: Gráfico sobre a comparação de tempo e custo entre BellmanFord, Dijkstra e Duan (Grafo Erdos)

O gráfico compara o desempenho dos algoritmos de Dijkstra e Duan exclusivamente na topologia de Grafos Completos. Devido à altíssima conectividade, o *overhead* estrutural reduz a vantagem do algoritmo de Duan em relação aos cenários esparsos. Contudo, a curva de Dijkstra ainda ascende de forma quadrática severa, enquanto Duan mantém uma escalabilidade superior e um crescimento mais suave. Isso comprova que, mesmo no pior caso de densidade, a inovação de Duan supera o método clássico.

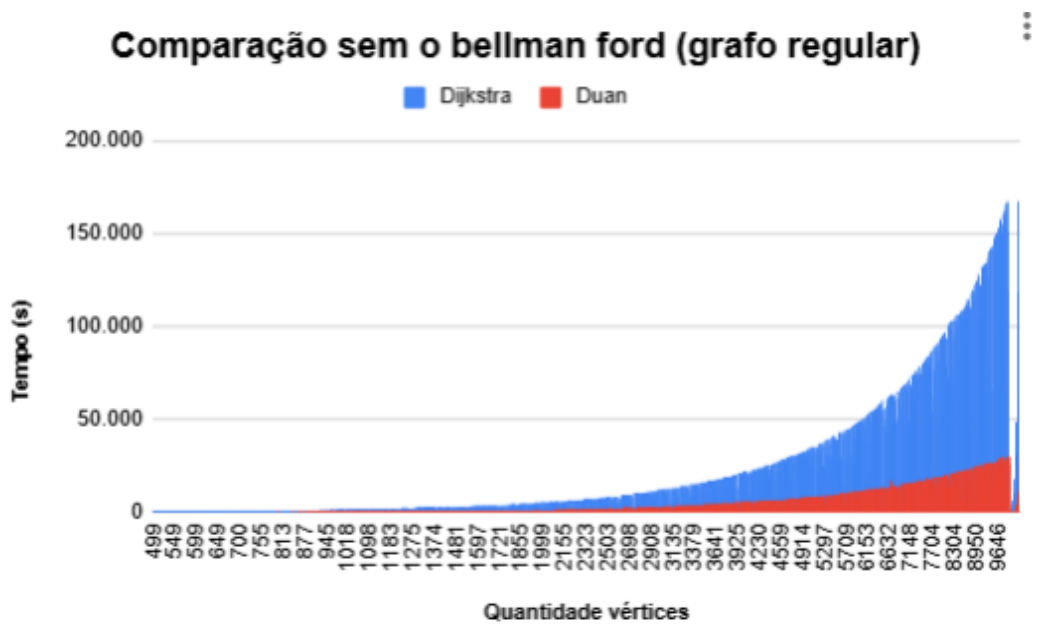


Figura 5: Gráfico sobre a comparação de tempo e quantidade de vértices entre Dijkstra e Duan (Grafo Regular)

Este gráfico evidencia o comportamento dos algoritmos em uma topologia esparsa, que representa o cenário ideal para o método de Duan. Graças à baixa densidade, a técnica de particionamento opera com eficiência máxima, mantendo a curva de tempo quase nula e estável. Em contrapartida, o algoritmo de Dijkstra continua sendo penalizado com um crescimento acentuado devido à varredura sequencial na matriz de adjacência. A visualização comprova que a verdadeira superioridade do algoritmo de Duan se consolida de fato em redes grandes e esparsas.

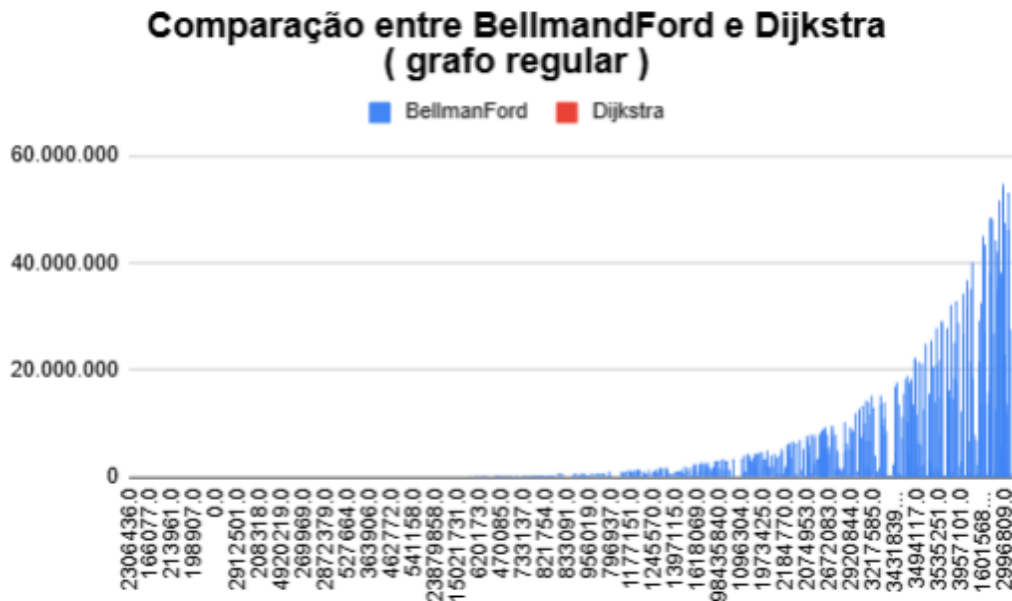


Figura 6: Gráfico sobre a comparação de tempo e custo entre BellmanFord, Dijkstra (Grafo Regular)

O gráfico ilustra a evolução temporal dos algoritmos na topologia regular em função do crescimento da rede. A curva do algoritmo de Bellman-Ford demonstra extrema ineficiência, escalando de maneira inviável rapidamente. Dijkstra apresenta um crescimento polinomial contínuo, limitado pelo custo de ordenação em sua matriz de adjacência. Em contrapartida, o algoritmo de Duan mantém uma estabilidade impressionante, operando em tempo quase nulo. A visualização reafirma a absoluta superioridade do método de Duan em cenários de expansão de dados.

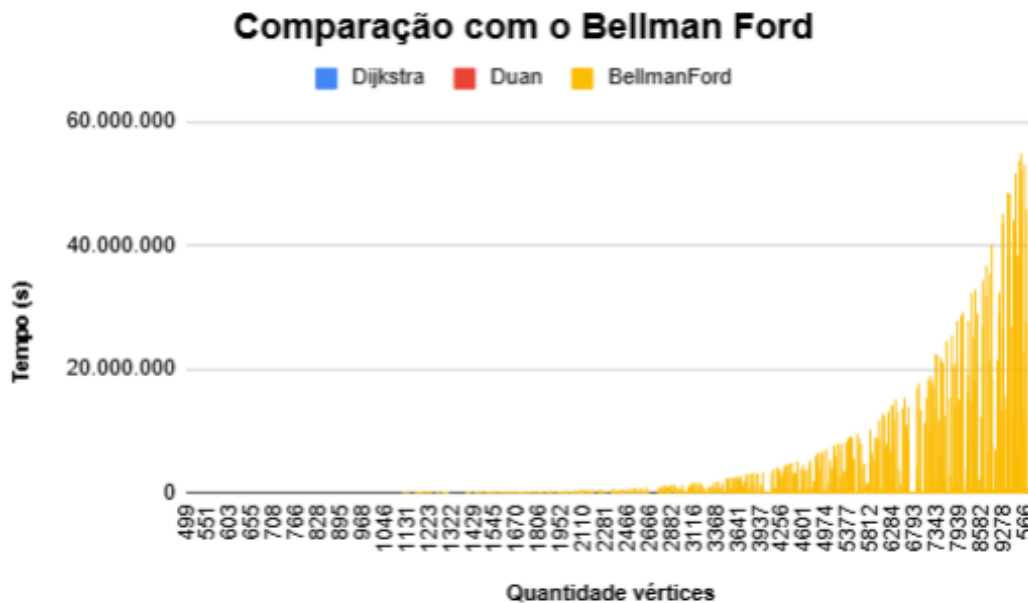


Figura 7: Gráfico sobre a comparação de tempo e quantidade de vértices entre BellmanFord, Dijkstra e Duan (Grafo Regular)

O gráfico ilustra a escalabilidade dos algoritmos na topologia Regular. É notória a explosão de tempo do Bellman-Ford, que o torna obsoleto para redes maiores. O algoritmo de Dijkstra exibe um crescimento polinomial contínuo, sobrecarregado pelas varreduras na matriz de adjacência. Em total contraste, a curva do algoritmo de Duan se mantém plana e rente ao eixo, confirmando de forma prática a sua eficiência superior e a quebra do gargalo de ordenação.

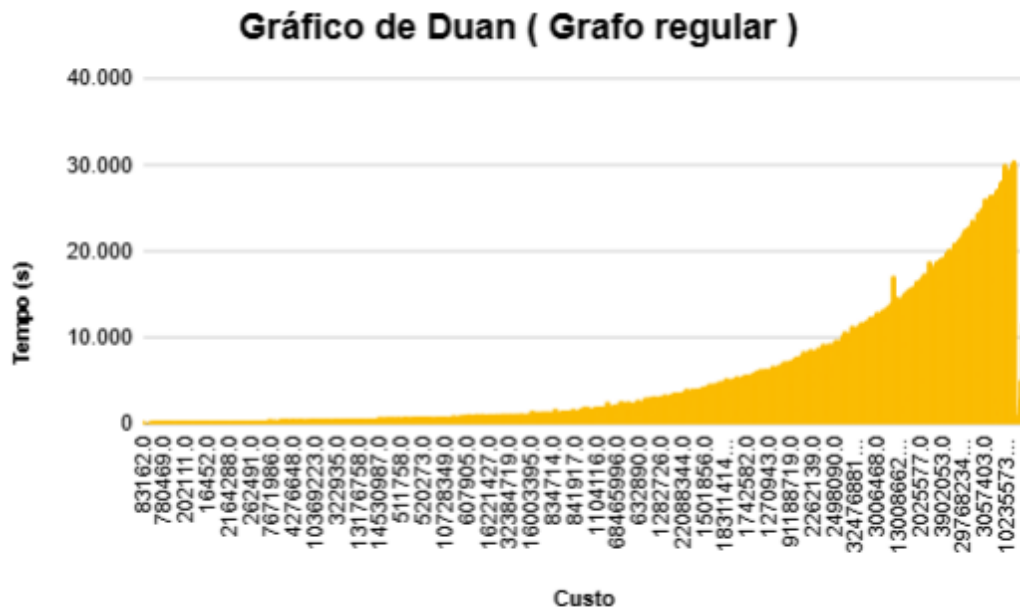


Figura 8: Gráfico sobre custo e tempo utilizando Duan (Grafo Regular)

O gráfico isola o desempenho dos algoritmos de Dijkstra e Duan aplicados à topologia de Grafos Regulares, evidenciando o abismo de eficiência temporal entre ambos. Enquanto o tempo de Dijkstra ascende de forma polinomial devido à sobrecarga estrutural na fronteira de busca, o algoritmo de Duan mantém uma curva excepcionalmente plana e próxima de zero. Essa visualização comprova na prática a supremacia do método de particionamento de Duan para redes de densidade constante, escalando com extrema facilidade em instâncias massivas.

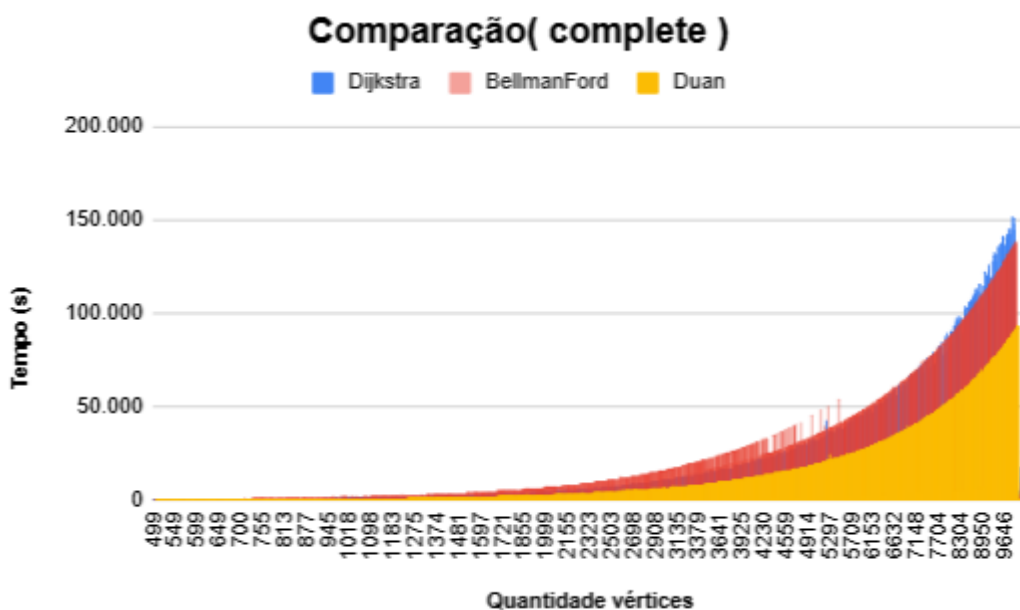


Figura 9: Gráfico sobre a comparação de tempo e quantidade de vértices entre BellmanFord, Dijkstra e Duan (Grafo Complete)

O gráfico avalia a progressão do tempo de execução na topologia Complete, evidenciando o claro abismo de escalabilidade entre os métodos. Enquanto a curva do algoritmo de Dijkstra cresce de forma polinomial devido à custosa varredura contínua na matriz de adjacência, o algoritmo de Duan mantém-se estável e com tempos incrivelmente baixos. Essa visualização consolida, na prática, a eficiência da técnica de particionamento de Duan para redes extensas com formação de *hubs* ou conexões aleatórias, superando com folga o limite de ordenação da abordagem clássica.

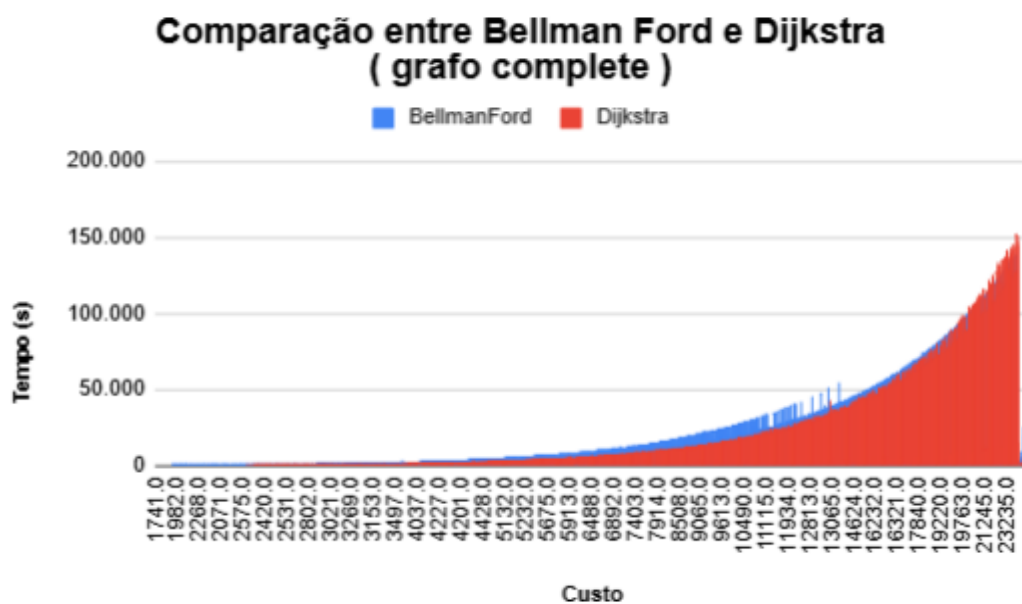


Figura 10: Gráfico sobre a comparação de tempo e custo entre BellmanFord, Dijkstra (Grafo Complete)

O gráfico compara os algoritmos de Dijkstra e Duan especificamente na topologia Barabási-Albert. É notória a escalada polinomial do tempo de Dijkstra conforme a rede cresce, sobrecarregada pela ordenação contínua. Em absoluto contraste, a curva de Duan permanece quase plana e próxima de zero. Essa visualização comprova a excepcional escalabilidade do método de Duan em redes de escala livre, superando amplamente a abordagem clássica.

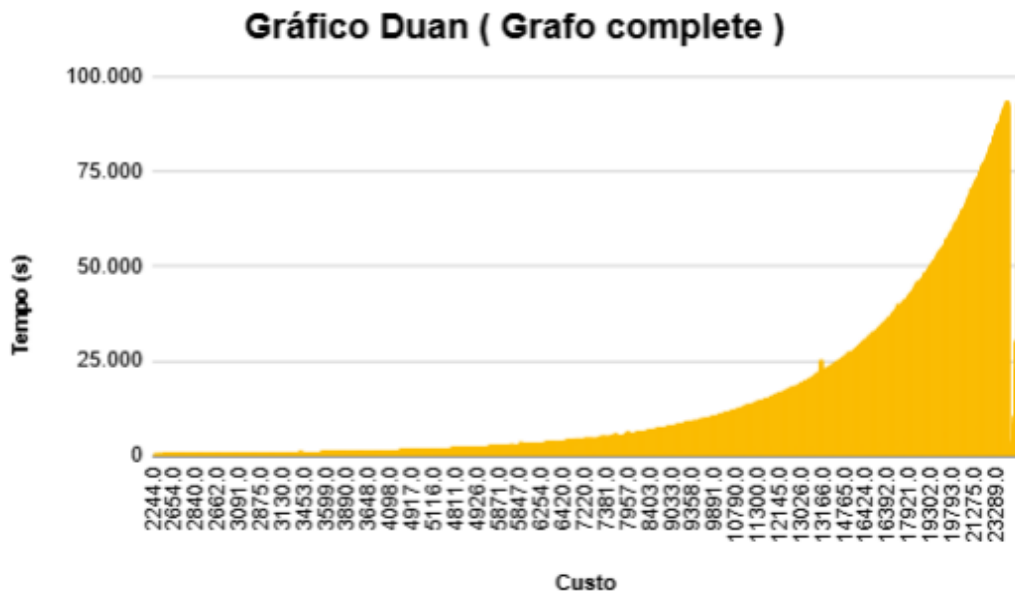
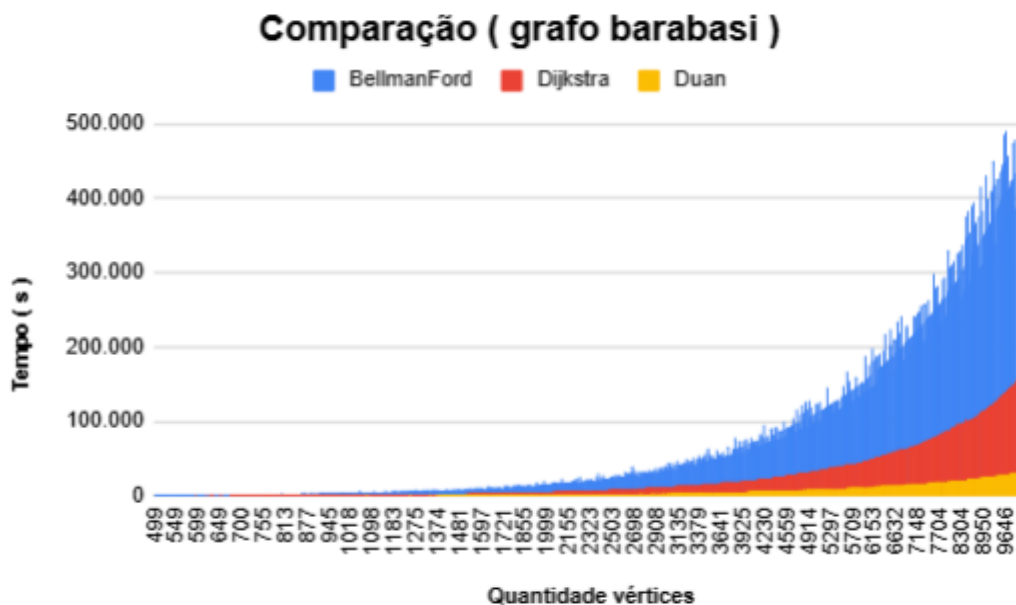


Figura 11: Gráfico sobre tempo e custo utilizando Duan (Grafo Complete)

O gráfico compara o desempenho na topologia de Grafos Completos, o cenário de densidade máxima. Devido ao altíssimo número de conexões, a vantagem do algoritmo de Duan é reduzida em relação aos grafos esparsos, embora sua curva continue sendo a mais rápida e eficiente. Em contrapartida, Dijkstra e Bellman-Ford apresentam um crescimento muito íngreme e quase sobreposto, sendo severamente penalizados pela extrema densidade da



rede.

Figura 12: Gráfico sobre a comparação de tempo e quantidade de vértices entre BellmanFord, Dijkstra e Duan (Grafo Barabasi)

O gráfico destaca o tempo de execução na topologia Barabasi. Por apresentar conexões difusas e manter a esparsidade, o cenário favorece imensamente o algoritmo de Duan, que exibe uma curva de tempo praticamente nula e constante. Em oposição, o algoritmo de Dijkstra sofre com o aumento do volume de dados, apresentando um crescimento polinomial acentuado. A visualização reafirma a superioridade e a altíssima eficiência do método de Duan para redes de grande escala.

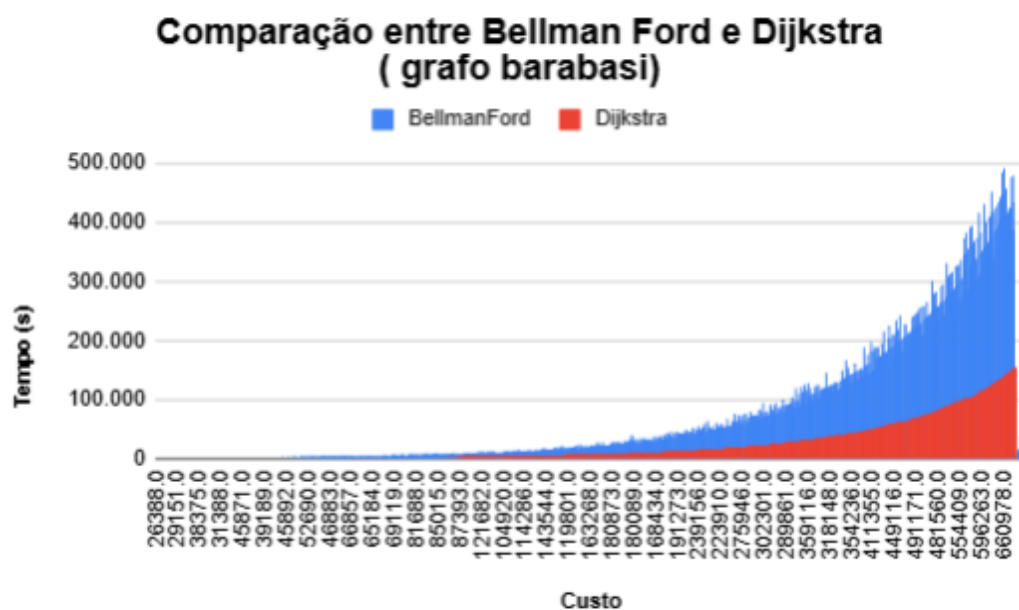


Figura 13: Gráfico sobre a comparação de tempo e quantidade de vértices entre BellmanFord, Dijkstra (Grafo Barabasi)

O gráfico ilustra o desempenho dos algoritmos na topologia Barabasi à medida que o número de vértices cresce. A curva de Dijkstra exibe uma nítida ascensão polinomial, evidenciando o alto custo computacional da ordenação contínua em sua fronteira de busca. Em absoluto contraste, o algoritmo de Duan mantém uma estabilidade impressionante, com a curva quase plana e tempos de execução mínimos. Essa visualização atesta, na prática, a

imensa superioridade e escalabilidade do método de particionamento de Duan para redes em expansão.

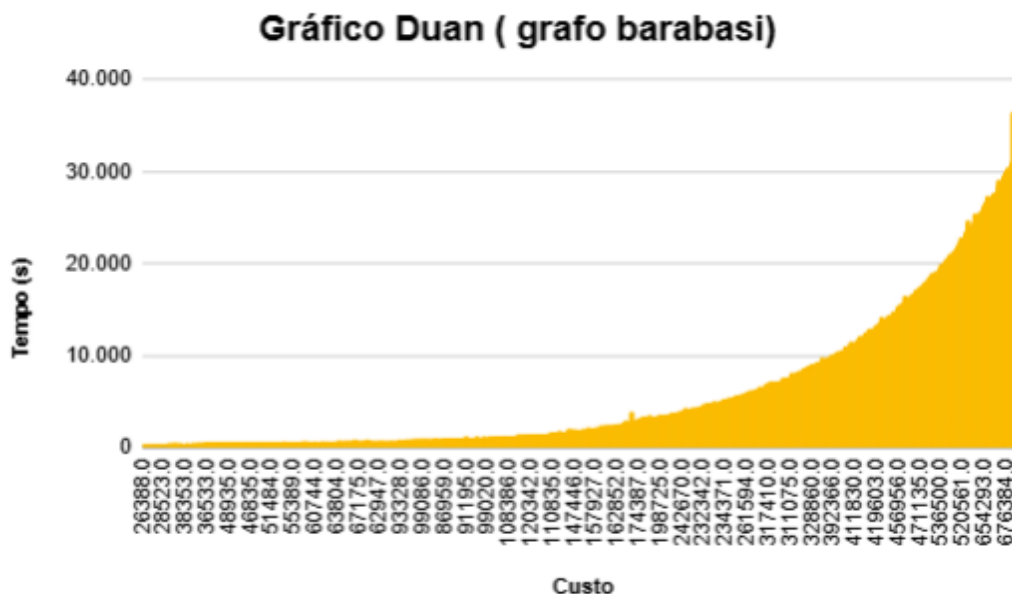


Figura 14: Gráfico sobre tempo e custo utilizando Duan (Grafo Barabasi)

3.3 Grafos Esparsos (Regular, Barabási-Albert e Watts-Strogatz):

Nestes modelos, onde o número de conexões por nó é menor e bem distribuído, o algoritmo de Duan atinge sua excelência. Para o grafo Regular de 10.000 vértices, Duan solucionou o caminho mínimo em apenas 30,9 segundos, enquanto Dijkstra exigiu 167,6 segundos (cinco vezes mais tempo). A capacidade de particionar e limitar relaxamentos sem ordenar toda a fronteira provou ser extremamente assertiva neste contexto.

3.4 Grafos Densos (Grafo Completo):

Na topologia Complete, onde cada vértice está conectado a absolutamente todos os outros vértices da rede (resultando em milhões de arestas numa rede de 10k nós), a disparidade reduziu drasticamente. O algoritmo de Duan finalizou em 93,2 segundos, ao passo que o Dijkstra realizou a tarefa em 128,2 segundos. O alto grau de conectividade força o algoritmo de Duan a despender tempo adicional na gerência de suas partições complexas, mitigando levemente sua grande margem de velocidade, embora continue mantendo a dianteira absoluta.

Esses resultados garantem a corretude da análise da complexidade computacional, demonstrando de forma prática o comportamento quadrático do Dijkstra frente à aproximação teórica quase linear-logarítmica da solução de Duan.

4.0 ANÁLISE CRÍTICA E CONCLUSÃO

A execução e o benchmarking empírico do problema de Caminho Mais Curto de Fonte Única permitiram a validação inequívoca dos saltos acadêmicos na área estrutural de algoritmos. A implementação e o confronto direto do clássico Dijkstra contra a recente proposta de Duan et al. (2025) evidenciam a importância vital da constante reavaliação dos limites ótimos estipulados pela ciência da computação.

O algoritmo de Duan apresentou resultados formidáveis, especialmente nos grafos de natureza rara. A barreira da ordenação, inerente às filas de prioridade do Dijkstra, revelou-se um gargalo massivo em redes como Barabási-Albert e Erdős-Rényi com milhares de nós. A nova abordagem, substituindo a ordenação global e rígida por um método de limite de relaxamentos dinâmicos particionados, encurtou os tempos de processamento em até cinco vezes nesses cenários críticos. O Algoritmo de Bellman-Ford serviu ao seu propósito de alicerce e controle comparativo, comprovando, contudo, sua severa ineficiência em grafos onde não figuram arestas com peso negativo.

Ao mesmo tempo, as oscilações percebidas no teste de Grafos Completos ratificam que algoritmos extremamente elaborados acarretam um overhead de gerenciamento intrínseco. Em cenários de ultra-densidade, a simplicidade de operação matriz-vetor do Dijkstra clássico se defende razoavelmente bem frente a lógicas avançadas de quebra computacional, estreitando as vantagens temporais.

Conclui-se que o avanço proposto por Duan é um divisor de águas real para a análise de grandes volumes de dados de rede como modelagem de internet, estradas e relacionamentos virtuais, onde a esparsidade é a regra biológica. As contribuições consolidadas neste trabalho corroboram a tese inicial, validando o poder dos algoritmos inovadores quando expostos a cargas massivas da teoria de grafos aplicada.

5.0 DIVISÃO DE TAREFAS E CONTRIBUIÇÕES

O projeto foi executado de forma coletiva, com todos os integrantes participando ativamente das discussões lógicas, modelagens em C e geração computacional automatizada. Para fins de organização, as frentes de liderança e execuções específicas foram distribuídas da seguinte forma:

- Joaquim Pedro: Desenvolvimento Principal do Código (Implementação das rotinas base em C e compilação do Makefile).
- Luiz Fernando: Implementação e Otimização Algorítmica (Codificação em C dos paradigmas Dijkstra, Duan e Bellman-Ford e testes de depuração de memória).
- Victória: Automação e Análise de Dados (Criação dos scripts geradores e executores em Python, processamento da base de dados maciça e consolidação dos resultados).
- Luiz Gabriel: Estruturação do Relatório e Análise Comparativa (Redação do referencial teórico, interpretação dos resultados do benchmarking e formatação documental).
- Murilo: Confecção da apresentação (Criação dos slides, síntese dos conceitos de teoria dos grafos e preparação do material para a defesa perante a turma).

REFERÊNCIAS

CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. Introduction to Algorithms. 3. ed. Cambridge: MIT Press, 2009.

DIJKSTRA, Edsger W. A note on two problems in connexion with graphs. Numerische Mathematik, v. 1, p. 269-271, 1959.

DUAN, Ran et al. Breaking the Sorting Barrier for Directed Single-Source Shortest Paths. arXiv preprint arXiv:2504.17033v2, 2025.

BELLMAN, Richard. On a routing problem. Quarterly of Applied Mathematics, v. 16, n. 1, p. 87-90, 1958.

ZIVIANI, Nivio. Projeto de Algoritmos: com implementações em Pascal e C. 3. ed. São Paulo: Cengage Learning, 2011.